



【金九银十】Vue 面试全流程解析

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 25年下半年面试真题预测及解析](#)
- [📖 阿里/字节面试高频真题解析与9月考点预测](#)
- [📖 阿里&字节9月面试高频真题预测](#)

2. Vue 高级用法：面试切入点详解

2.1 Vue2 Vue3双向绑定 对比

2.3 vue生命周期

Vue 2 通过 `Object.defineProperty`

- 监听不到对象属性的删除和添加，需要借助 `$set` 和 `$delete` 方法。
- 对于数组的API方法无法监听，vue2中对于数组的方法进行了包裹。
- 数据变化时通知依赖的Watcher 更新视图，需要递归遍历对象来设置getter 和setter，浪费性能。

Vue 3 采用Proxy 监听整个对象，无需递归

2.2 Vue3 比 Vue2 多了什么

- 性能提升:
 - 优化虚拟DOM (Virtual DOM) Vue 3 对虚拟DOM进行了优化，例如静态节点提升、静态props优化等，减少了不必要的DOM操作，提高了渲染性能
 - Vue 3 Proxy -> Vue 2 的Object.defineProperty
 - 碎片(Fragments) Vue 3 支持碎片，允许组件返回多个根节点，也减少了不必要的DOM层级，优化了渲染性能
 - Vue 3 支持tree shaking
 - vue 3 静态标记基础上，支持最长递增子序列
- 响应式系统:
 - Vue 3 Proxy -> Vue 2 的Object.defineProperty
- 代码组织:
 - Vue 3 引入了Composition API，允许开发者更好地组织和复用组件逻辑，使得代码更易于维护和测试。
 - 源码层面上优化 Vue 3 采用模块化设计，将各个组件功能分离，增强了应用程序的灵活性和可扩展性。
- TypeScript 支持:
 - Vue 3 原生支持TypeScript

2.4 Ref reactive 区别

特性	ref	reactive
适用数据类型	基本类型和对象	只适用于对象（包括数组）
访问方式	需要通过 <code>.value</code> 访问	直接访问属性
响应式原理	使用对象包装 + getter/setter	使用 Proxy 代理
模板自动解包	自动解包（无需 <code>.value</code> ）	直接使用
重新赋值能力	可以整个替换	不能替换整个对象
TypeScript 支持	更好的类型推断	嵌套属性类型推断较复杂

Code block

```
1 // 简化的 ref 实现
2 function ref(value) {
```

1. 创建(Creation):

- `beforeCreate` : 在实例被创建之初，数据观测(data observer) 和事件(event/watcher) 都尚未初始化。
- `created` : 实例已经创建完成，数据观测(data observer)、属性和方法的运算，以及watch/event 事件回调都已设置完成，但还没有挂载到DOM 上。

2. 2. 挂载(Mounting):

- `beforeMount` : 在模板编译/渲染成DOM 之前被调用。
- `mounted` : 实例已经挂载到DOM 上，此时可以访问真实的DOM 节点。

3. 3. 更新(Updating):

- `beforeUpdate` : 在数据发生变化，DOM 重新渲染之前被调用。
- `updated` : 组件DOM 已经更新完成。

4. 4. 销毁(Destruction):

- `beforeDestroy` : 实例销毁之前调用。在这一步，实例仍然完全可用。
- `destroyed` : 实例销毁后调用。调用后，Vue 实例的所有指令都解除绑定，所有事件监听器都被移除，所有的子实例也都会被销毁

vue3 新增 `renderTracked` `renderTracked`

Code block

```
1 // 简化的 reactive 实现
2 function reactive(target) {
3   return new Proxy(target, {
4     get(target, key, receiver) {
5       track(target, key); // 依赖收集
6       return Reflect.get(target, key,
7         receiver);
8     },
9     set(target, key, value, receiver) {
10      Reflect.set(target, key, value,
11        receiver);
12      trigger(target, key); // 触发更新
13      return true;
14    }
15  });
16 }
```

```

3     const refObject = {
4       get value() {
5         track(refObject, 'value'); // 依赖收集
6         return value;
7       },
8       set value(newVal) {
9         value = newVal;
10        trigger(refObject, 'value'); // 触发更新
11      }
12    };
13    return refObject;
14  }

```

```

13    });
14  }

```

1. Ref -> reactive: reactive
2. Reactive -> ref: toRefs

使用建议

- 对于简单值，`ref` 比 `reactive` 更轻量
- 对于复杂对象，`reactive` 比深度 `ref` 更高效
- 在模板中使用大量 `ref` 会导致更多 `.value` 访问（虽然模板会自动解包）

2.5 Vue nextTick 实现

在 DOM 更新周期后执行延迟回调，主要基于 JavaScript 的微任务队列机制

Vue 3 使用 Promise 作为 `nextTick` 的实现基础，原因包括1：

1. 微任务优先级高于宏任务（如 `setTimeout`），能更早执行
2. 现代浏览器广泛支持 Promise
3. 可以形成良好的 Promise 链式调用

Code block

```

1  const resolvedPromise = Promise.resolve() as
    Promise<any>
2  let currentFlushPromise: Promise<void> | null
    = null
3
4  export function nextTick<T = void>(<
5    this: T,
6    fn?: (this: T) => void
7  ): Promise<void> {
8    const p = currentFlushPromise ||
    resolvedPromise
9    return fn ? p.then(this ? fn.bind(this) :
    fn) : p
10 }

```

Vue 2 中 `nextTick` 的实现更加复杂，有一个降级策略：

1. 优先使用 Promise
2. 不支持 Promise 时回退到 MutationObserver
3. 再不支持则使用 `setImmediate`
4. 最后使用 `setTimeout`

2.6 Vue 通信

1. 父子组件通信

- **Props**：父组件通过 `props` 向子组件传递数据。子组件通过 `props` 接收数据。
- `$emit` 和事件 `$on`
- `$parent` 和 `$children`：`$parent` 访问父组件实例，`$children` 访问所有子组件实例。不推荐直接访问组件实例，因为这会破坏组件的封装性。
- `provide` 和 `inject`
- `$attrs` 和 `$listeners`：

2. 兄弟组件通信

- `eventBus`：创建一个空的Vue实例作为事件总线，通过 `bus.$emit` 触发事件，`bus.$on` 监听事件。这种方式简单，但适用于小型应用，对于大型应用，更推荐使用
- `Vuex`：Vuex 是一个专为Vue.js 应用程序开发的状态管理模式。它采用集中式存储管理应用的所有组件的状态，并以相应的规则保证状态的唯一性。
- 3. 跨层级组件通信：
 - `eventBus`：同上，适用于小型应用。
 - `Vuex`：同上，适用于大型应用。
 - `provide` 和 `inject`：同上。

2.7 Vue 状态管理

方案	适用场景	优点	缺点	复杂度	适用规模
组件内状态 (data)	单个组件内部状态	简单直接，无需额外依赖	难以跨组件共享	低	小型应用
Props/Events	父子组件通信	Vue 原生支持，简单明了	多层嵌套时繁琐	低	小型应用
Provide/Inject	深层嵌套组件通信	避免 prop 逐层传递	非响应式(默认)，组织性较差	中	中小型应用
Event Bus	任意组件间通信	简单全局通信方案	难以追踪调试，可能内存泄漏	中	不推荐使用
Vuex	中大型应用集中式状态	集中管理，调试工具支持，可预测状态变化	相对繁琐，学习曲线	高	中大型应用
Pinia	Vue 2/3 应用状态管理	更简单的 API，TypeScript 支持，模块化	较新，生态较小	中高	各种规模
Composition API (reactive/ref)	组合式逻辑复用	灵活，逻辑组合方便	需要良好组织	中	各种规模

3. Vue 进阶：Vue源码解析

 vue3 核心源码.zip

545.86KB



官方地址：<https://github.com/vuejs/core/tree/main/packages>

基本核心模块目录结构如下：

Code block

```
1  |─compiler-core
2  |  | package.json
3  |  |
4  |  |─src
5  |  |  | ast.ts
6  |  |  | codegen.ts
7  |  |  | compile.ts
8  |  |  | index.ts
9  |  |  | parse.ts
10 |  |  | runtimeHelpers.ts
11 |  |  | transform.ts
12 |  |  | utils.ts
13 |  |  |
14 |  |  |─transforms
15 |  |  |  | transformElement.ts
16 |  |  |  | transformExpression.ts
17 |  |  |  | transformText.ts
18 |  |  |
19 |  |─__tests__
```

```
20 | | codegen.spec.ts
21 | | parse.spec.ts
22 | | transform.spec.ts
23 | |
24 | | └─__snapshots__
25 | |     codegen.spec.ts.snap
26 | |
27 | └─reactivity
28 | |   package.json
29 | |
30 | | └─src
31 | | |   baseHandlers.ts
32 | | |   computed.ts
33 | | |   dep.ts
34 | | |   effect.ts
35 | | |   index.ts
36 | | |   reactive.ts
37 | | |   ref.ts
38 | | |
39 | | | └─__tests__
40 | | | |   computed.spec.ts
41 | | | |   dep.spec.ts
42 | | | |   effect.spec.ts
43 | | | |   reactive.spec.ts
44 | | | |   readonly.spec.ts
45 | | | |   ref.spec.ts
46 | | | |   shallowReadonly.spec.ts
47 | | |
48 | └─runtime-core
49 | |   package.json
50 | |
51 | | └─src
52 | | |   .pnpm-debug.log
53 | | |   apiInject.ts
54 | | |   apiWatch.ts
55 | | |   component.ts
56 | | |   componentEmits.ts
57 | | |   componentProps.ts
58 | | |   componentPublicInstance.ts
59 | | |   componentRenderUtils.ts
60 | | |   componentSlots.ts
61 | | |   createApp.ts
62 | | |   h.ts
63 | | |   index.ts
64 | | |   renderer.ts
65 | | |   scheduler.ts
66 | | |   vnode.ts
67 | | |
68 | | | └─helpers
69 | | | |   renderSlot.ts
70 | | |
71 | | | └─__tests__
72 | | | |   apiWatch.spec.ts
73 | | | |   componentEmits.spec.ts
74 | | | |   rendererComponent.spec.ts
75 | | | |   rendererElement.spec.ts
76 | | |
77 | └─runtime-dom
78 | |   package.json
79 | |
80 | | └─src
81 | | |   index.ts
```

```

82 |
83 | └─runtime-test
84 |   └─src
85 |       index.ts
86 |       nodeOps.ts
87 |       patchProp.ts
88 |       serialize.ts
89 |
90 | └─shared
91 |   │ package.json
92 |   │
93 |   └─src
94 |       index.ts
95 |       shapeFlags.ts
96 |       toDisplayString.ts

```

3.1 compiler-core

Vue3的编译核心，核心作用就是将字符串转换成 抽象对象语法树AST；

3.1.1 目录结构

Code block

```

1  | └─src
2  |   │ ast.ts           // ts类型定义, 比如type, enum, interface等
3  |   │ codegen.ts       // 将生成的ast转换成render字符串
4  |   │ compile.ts      // compile统一执行逻辑, 有一个 baseCompile , 用来编译模板文件的
5  |   │ index.ts        // 入口文件
6  |   │ parse.ts        // 将模板字符串转换成 AST
7  |   │ runtimeHelpers.ts // 生成code的时候的定义常量对应关系
8  |   │ transform.ts     // 处理 AST 中的 vue 特有语法
9  |   │ utils.ts
10 |   │
11 |   └─transforms
12 |       transformElement.ts
13 |       transformExpression.ts
14 |       transformText.ts
15 |
16 | └─__tests__           // 测试用例
17 |     │ codegen.spec.ts
18 |     │ parse.spec.ts
19 |     │ transform.spec.ts
20 |     │
21 |     └─__snapshots__
22 |         codegen.spec.ts.snap
23 |

```

3.1.2 compile逻辑

Code block

```

1  // src/index.ts
2  export { baseCompile } from './compile';
3
4  // src/compiler.ts
5  import { generate } from './codegen';
6  import { baseParse } from './parse';
7  import { transform } from './transform';
8  import { transformExpression } from './transforms/transformExpression';

```



```

9  import { transformElement } from "../transforms/transformElement";
10 import { transformText } from "../transforms/transformText";
11
12 export function baseCompile(template, options) {
13   // 1. 先把 template 也就是字符串 parse 成 ast
14   const ast = baseParse(template);
15   // 2. 给 ast 加点料 (- -#)
16   transform(
17     ast,
18     Object.assign(options, {
19       nodeTransforms: [transformElement, transformText, transformExpression],
20     })
21   );
22
23   // 3. 生成 render 函数代码
24   return generate(ast);
25 }

```

- baseParse

Code block

```

1  export function baseParse(content: string) {
2    const context = createParserContext(content);
3    return createRoot(parseChildren(context, []));
4  }
5
6  function createParserContext(content) {
7    console.log("创建 parserContext");
8    return {
9      source: content,
10    };
11  }
12
13  function createRoot(children) {
14    return {
15      type: NodeTypes.ROOT,
16      children,
17      helpers: [],
18    };
19  }
20
21  function parseChildren(context, ancestors) {
22    console.log("开始解析 children");
23    const nodes: any = [];
24
25    while (!isEnd(context, ancestors)) {
26      let node;
27      const s = context.source;
28
29      if (startsWith(s, "{")) {
30        // 看看如果是 {{ 开头的话, 那么就是一个插值, 那么去解析他
31        node = parseInterpolation(context);
32      } else if (s[0] === "<") {
33        if (s[1] === "/" ) {
34          // 这里属于 edge case 可以不用关心
35          // 处理结束标签
36          if (/[a-z]/i.test(s[2])) {
37            // 匹配 </div>
38            // 需要改变 context.source 的值 -> 也就是需要移动光标
39            parseTag(context, TagType.End);

```

```

40         // 结束标签就以为这都已经处理完了，所以就可以跳出本次循环了
41         continue;
42     }
43     } else if (/[a-z]/i.test(s[1])) {
44         node = parseElement(context, ancestors);
45     }
46 }
47
48 if (!node) {
49     node = parseText(context);
50 }
51
52 nodes.push(node);
53 }
54
55 return nodes;
56 }

```

- transform

Code block

```

1  export function transform(root, options = {}) {
2      // 1. 创建 context
3
4      const context = createTransformContext(root, options);
5
6      // 2. 遍历 node
7      traverseNode(root, context);
8
9      createRootCodegen(root, context);
10
11     root.helpers.push(...context.helpers.keys());
12 }
13
14 function createTransformContext(root, options): any {
15     const context = {
16         root,
17         nodeTransforms: options.nodeTransforms || [],
18         helpers: new Map(),
19         helper(name) {
20             // 这里会收集调用的次数
21             // 收集次数是为了给删除做处理的，（当只有 count 为0 的时候才需要真的删除掉）
22             // helpers 数据会在后续生成代码的时候用到
23             const count = context.helpers.get(name) || 0;
24             context.helpers.set(name, count + 1);
25         },
26     };
27
28     return context;
29 }
30
31 function traverseNode(node: any, context) {
32     const type: NodeTypes = node.type;
33
34     // 遍历调用所有的 nodeTransforms
35     // 把 node 给到 transform
36     // 用户可以对 node 做处理
37     const nodeTransforms = context.nodeTransforms;
38     const exitFns: any = [];
39     for (let i = 0; i < nodeTransforms.length; i++) {

```



```

40     const transform = nodeTransforms[i];
41
42     const onExit = transform(node, context);
43     if (onExit) {
44         exitFns.push(onExit);
45     }
46 }
47
48 switch (type) {
49     case NodeTypes.INTERPOLATION:
50         // 插值的点，在于后续生成 render 代码的时候是获取变量的值
51         context.helper(TO_DISPLAY_STRING);
52         break;
53
54     case NodeTypes.ROOT:
55     case NodeTypes.ELEMENT:
56
57         traverseChildren(node, context);
58         break;
59
60     default:
61         break;
62 }
63
64
65
66 let i = exitFns.length;
67 // i-- 这个很巧妙
68 // 使用 while 是要比 for 快 (可以使用 https://jsbench.me/ 来测试一下)
69 while (i--) {
70     exitFns[i]();
71 }
72 }
73
74 function createRootCodegen(root: any, context: any) {
75     const { children } = root;
76
77     // 只支持有一个根节点
78     // 并且还是一个 single text node
79     const child = children[0];
80
81     // 如果是 element 类型的话，那么我们需要把它的 codegenNode 赋值给 root
82     // root 其实是个空的什么数据都没有的节点
83     // 所以这里需要额外的处理 codegenNode
84     // codegenNode 的目的是专门为了 codegen 准备的 为的就是和 ast 的 node 分离开
85     if (child.type === NodeTypes.ELEMENT && child.codegenNode) {
86         const codegenNode = child.codegenNode;
87         root.codegenNode = codegenNode;
88     } else {
89         root.codegenNode = child;
90     }
91 }
92

```

- generate

Code block

```

1  export function generate(ast, options = {}) {
2      // 先生成 context
3      const context = createCodegenContext(ast, options);

```

```

4   const { push, mode } = context;
5
6   // 1. 先生成 preambleContext
7
8   if (mode === "module") {
9     genModulePreamble(ast, context);
10  } else {
11    genFunctionPreamble(ast, context);
12  }
13
14  const functionName = "render";
15
16  const args = ["_ctx"];
17
18  // _ctx,aaa,bbb,ccc
19  // 需要把 args 处理成 上面的 string
20  const signature = args.join(", ");
21  push(`function ${functionName}(${signature}) {`);
22  // 这里需要生成具体的代码内容
23  // 开始生成 vnode tree 的表达式
24  push("return ");
25  genNode(ast.codegenNode, context);
26
27  push("}");
28
29  return {
30    code: context.code,
31  };
32 }

```

3.2 reactivity

负责Vue3中响应式实现的部分

3.2.1 目录结构

Code block

```

1  |—src
2  |   baseHandlers.ts // 基本处理逻辑
3  |   computed.ts // computed属性处理
4  |   dep.ts // effect对象存储逻辑
5  |   effect.ts // 依赖收集机制
6  |   index.ts // 入口文件
7  |   reactive.ts // 响应式处理逻辑
8  |   ref.ts // ref执行逻辑
9  |
10 |—__tests__ // 测试用例
11 |   computed.spec.ts
12 |   dep.spec.ts
13 |   effect.spec.ts
14 |   reactive.spec.ts
15 |   readonly.spec.ts
16 |   ref.spec.ts
17 |   shallowReadonly.spec.ts

```

3.2.2 reactivity逻辑

- index.ts

```

1 export {
  Code block
2   reactive,
3   readonly,
4   shallowReadonly,
5   isReadonly,
6   isReactive,
7   isProxy,
8 } from "./reactive";
9
10 export { ref, proxyRefs, unRef, isRef } from "./ref";
11
12 export { effect, stop, ReactiveEffect } from "./effect";
13
14 export { computed } from "./computed";
15

```

- reactive.ts

```

Code block
1 import {
2   mutableHandlers,
3   readonlyHandlers,
4   shallowReadonlyHandlers,
5 } from "./baseHandlers";
6
7 export const reactiveMap = new WeakMap();
8 export const readonlyMap = new WeakMap();
9 export const shallowReadonlyMap = new WeakMap();
10
11 export const enum ReactiveFlags {
12   IS_REACTIVE = "__v_isReactive",
13   IS_READONLY = "__v_isReadonly",
14   RAW = "__v_raw",
15 }
16
17 export function reactive(target) {
18   return createReactiveObject(target, reactiveMap, mutableHandlers);
19 }
20
21 export function readonly(target) {
22   return createReactiveObject(target, readonlyMap, readonlyHandlers);
23 }
24
25 export function shallowReadonly(target) {
26   return createReactiveObject(
27     target,
28     shallowReadonlyMap,
29     shallowReadonlyHandlers
30   );
31 }
32
33 export function isProxy(value) {
34   return isReactive(value) || isReadonly(value);
35 }
36
37 export function isReadonly(value) {
38   return !!value[ReactiveFlags.IS_READONLY];
39 }
40
41 export function isReactive(value) {

```

```

42 // 如果 value 是 proxy 的话
43 // 会触发 get 操作, 而在 createGetter 里面会判断
44 // 如果 value 是普通对象的话
45 // 那么会返回 undefined , 那么就需要转换成布尔值
46 return !!value[ReactiveFlags.IS_REACTIVE];
47 }
48
49 export function toRaw(value) {
50 // 如果 value 是 proxy 的话 , 那么直接返回就可以了
51 // 因为会触发 createGetter 内的逻辑
52 // 如果 value 是普通对象的话,
53 // 我们就应该返回普通对象
54 // 只要不是 proxy , 只要是得到了 undefined 的话, 那么就一定是普通对象
55 // TODO 这里和源码里面实现的不一样, 不确定后面会不会有问题
56 if (!value[ReactiveFlags.RAW]) {
57   return value;
58 }
59
60 return value[ReactiveFlags.RAW];
61 }
62
63 function createReactiveObject(target, proxyMap, baseHandlers) {
64 // 核心就是 proxy
65 // 目的是可以侦听到用户 get 或者 set 的动作
66
67 // 如果命中的话就直接返回就好了
68 // 使用缓存做的优化点
69 const existingProxy = proxyMap.get(target);
70 if (existingProxy) {
71   return existingProxy;
72 }
73
74 const proxy = new Proxy(target, baseHandlers);
75
76 // 把创建好的 proxy 给存起来,
77 proxyMap.set(target, proxy);
78 return proxy;
79 }
80

```

- ref.ts

Code block

```

1 import { trackEffects, triggerEffects, isTracking } from "../effect";
2 import { createDep } from "../dep";
3 import { isObject, hasChanged } from "@mini-vue/shared";
4 import { reactive } from "../reactive";
5
6 export class RefImpl {
7   private _rawValue: any;
8   private _value: any;
9   public dep;
10  public __v_isRef = true;
11
12  constructor(value) {
13    this._rawValue = value;
14    // 看看value 是不是一个对象, 如果是一个对象的话
15    // 那么需要用 reactive 包裹一下
16    this._value = convert(value);
17    this.dep = createDep();

```

```
18     }
19
20     get value() {
21         // 收集依赖
22         trackRefValue(this);
23         return this._value;
24     }
25
26     set value(newValue) {
27         // 当新的值不等于老的值的话,
28         // 那么才需要触发依赖
29         if (hasChanged(newValue, this._rawValue)) {
30             // 更新值
31             this._value = convert(newValue);
32             this._rawValue = newValue;
33             // 触发依赖
34             triggerRefValue(this);
35         }
36     }
37 }
38
39 export function ref(value) {
40     return createRef(value);
41 }
42
43 function convert(value) {
44     return isObject(value) ? reactive(value) : value;
45 }
46
47 function createRef(value) {
48     const refImpl = new RefImpl(value);
49
50     return refImpl;
51 }
52
53 export function triggerRefValue(ref) {
54     triggerEffects(ref.dep);
55 }
56
57 export function trackRefValue(ref) {
58     if (isTracking()) {
59         trackEffects(ref.dep);
60     }
61 }
62
63 // 这个函数的目的是
64 // 帮助解构 ref
65 // 比如在 template 中使用 ref 的时候, 直接使用就可以了
66 // 例如: const count = ref(0) -> 在 template 中使用的话 可以直接 count
67 // 解决方案就是通过 proxy 来对 ref 做处理
68
69 const shallowUnwrapHandlers = {
70     get(target, key, receiver) {
71         // 如果里面是一个 ref 类型的话, 那么就返回 .value
72         // 如果不是的话, 那么直接返回value 就可以了
73         return unRef(Reflect.get(target, key, receiver));
74     },
75     set(target, key, value, receiver) {
76         const oldValue = target[key];
77         if (isRef(oldValue) && !isRef(value)) {
78             return (target[key].value = value);
79         } else {
```

```

80     return Reflect.set(target, key, value, receiver);
81   }
82 },
83 };
84
85 // 这里没有处理 objectWithRefs 是 reactive 类型的时候
86 // TODO reactive 里面如果有 ref 类型的 key 的话, 那么也是不需要调用 ref.value 的
87 // (but 这个逻辑在 reactive 里面没有实现)
88 export function proxyRefs(objectWithRefs) {
89   return new Proxy(objectWithRefs, shallowUnwrapHandlers);
90 }
91
92 // 把 ref 里面的值拿到
93 export function unRef(ref) {
94   return isRef(ref) ? ref.value : ref;
95 }
96
97 export function isRef(value) {
98   return !!value.__v_isRef;
99 }
100

```

- effect

Code block

```

1  export function effect(fn, options = {}) {
2    const _effect = new ReactiveEffect(fn);
3
4    // 把用户传过来的值合并到 _effect 对象上去
5    // 缺点就是不是显式的, 看代码的时候并不知道有什么值
6    extend(_effect, options);
7    _effect.run();
8
9    // 把 _effect.run 这个方法返回
10   // 让用户可以自行选择调用的时机 (调用 fn)
11   const runner: any = _effect.run.bind(_effect);
12   runner.effect = _effect;
13   return runner;
14 }
15
16 export function stop(runner) {
17   runner.effect.stop();
18 }

```

- computed

Code block

```

1  import { createDep } from './dep';
2  import { ReactiveEffect } from './effect';
3  import { trackRefValue, triggerRefValue } from './ref';
4
5  export class ComputedRefImpl {
6    public dep: any;
7    public effect: ReactiveEffect;
8
9    private _dirty: boolean;
10   private _value
11

```



```

12   constructor(getter) {
13     this._dirty = true;
14     this.dep = createDep();
15     this.effect = new ReactiveEffect(getter, () => {
16       // scheduler
17       // 只要触发了这个函数说明响应式对象的值发生了改变
18       // 那么就解锁，后续在调用 get 的时候就会重新执行，所以会得到最新的值
19       if (this._dirty) return;
20
21       this._dirty = true;
22       triggerRefValue(this);
23     });
24   }
25
26   get value() {
27     // 收集依赖
28     trackRefValue(this);
29     // 锁上，只可以调用一次
30     // 当数据改变的时候才会解锁
31     // 这里就是缓存实现的核心
32     // 解锁是在 scheduler 里面做的
33     if (this._dirty) {
34       this._dirty = false;
35       // 这里执行 run 的话，就是执行用户传入的 fn
36       this._value = this.effect.run();
37     }
38
39     return this._value;
40   }
41 }
42
43 export function computed(getter) {
44   return new ComputedRefImpl(getter);
45 }
46

```

3.3 runtime-core

运行的核心流程，其中包括初始化流程和更新流程

3.3.1 目录结构

Code block

```

1  |—src
2  |   | apiInject.ts      // 提供provider和inject
3  |   | apiWatch.ts       // 提供watch
4  |   | component.ts       // 创建组件实例
5  |   | componentEmits.ts  // 执行组件props 里面的 onXXX 的函数
6  |   | componentProps.ts  // 获取组件props
7  |   | componentPublicInstance.ts // 组件通用实例上的代理,如$el,$emit等
8  |   | componentRenderUtils.ts // 判断组件是否需要重新渲染的工具类
9  |   | componentSlots.ts  // 组件的slot
10 |   | createApp.ts        // 根据跟组件创建应用
11 |   | h.ts                // 创建节点
12 |   | index.ts            // 入口文件
13 |   | renderer.ts         // 渲染机制,包含diff
14 |   | scheduler.ts        // 触发更新机制
15 |   | vnode.ts            // vnode节点
16 |   |
17 | —helpers

```

```

18 |         renderSlot.ts          // 插槽渲染实现
19 |
20 | └─ __tests__                  // 测试用例
21 |     apiWatch.spec.ts
22 |     componentEmits.spec.ts
23 |     rendererComponent.spec.ts
24 |     rendererElement.spec.ts

```

3.3.2 runtime核心逻辑

- provide/inject

Code block

```

1  import { getCurrentInstance } from "../component";
2
3  export function provide(key, value) {
4      const currentInstance = getCurrentInstance();
5
6      if (currentInstance) {
7          let { provides } = currentInstance;
8
9          const parentProvides = currentInstance.parent?.provides;
10
11         // 这里要解决一个问题
12         // 当父级 key 和 爷爷级别的 key 重复的时候，对于子组件来讲，需要取最近的父级别组件的值
13         // 那这里的解决方案就是利用原型链来解决
14         // provides 初始化的时候是在 createComponent 时处理的，当时是直接把 parent.provides 赋值给组件的 provides
15         // 的
16         // 所以，如果说这里发现 provides 和 parentProvides 相等的话，那么就说明是第一次做 provide(对于当前组件来讲)
17         // 我们就可以把 parent.provides 作为 currentInstance.provides 的原型重新赋值
18         // 至于为什么不在 createComponent 的时候做这个处理，可能的好处是在这里初始化的话，是有个懒执行的效果（优化点，只
19         // 有需要的时候在初始化）
20
21         if (parentProvides === provides) {
22             provides = currentInstance.provides = Object.create(parentProvides);
23         }
24
25         provides[key] = value;
26     }
27 }
28
29 export function inject(key, defaultValue) {
30     const currentInstance = getCurrentInstance();
31
32     if (currentInstance) {
33         const provides = currentInstance.parent?.provides;
34
35         if (key in provides) {
36             return provides[key];
37         } else if (defaultValue) {
38             if (typeof defaultValue === "function") {
39                 return defaultValue();
40             }
41             return defaultValue;
42         }
43     }
44 }

```

- watch

Code block

```
1  import { ReactiveEffect } from "@mini-vue/reactivity";
2  import { queuePreFlushCb } from "../scheduler";
3
4  // Simple effect.
5  export function watchEffect(effect) {
6    doWatch(effect);
7  }
8
9  function doWatch(source) {
10   // 把 job 添加到 pre flush 里面
11   // 也就是在视图更新完成之前进行渲染（待确认？）
12   // 当逻辑执行到这里的时候 就已经触发了 watchEffect
13   const job = () => {
14     effect.run();
15   };
16
17   // 这里用 scheduler 的目的就是在更新的时候
18   // 让回调可以在 render 前执行 变成一个异步的行为（这里也可以通过 flush 来改变）
19   const scheduler = () => queuePreFlushCb(job);
20
21   const getter = () => {
22     source();
23   };
24
25   const effect = new ReactiveEffect(getter, scheduler);
26
27   // 这里执行的就是 getter
28   effect.run();
29 }
30
```

- component创建

Code block

```
1  export function createComponentInstance(vnode, parent) {
2    const instance = {
3      type: vnode.type,
4      vnode,
5      next: null, // 需要更新的 vnode, 用于更新 component 类型的组件
6      props: {},
7      parent,
8      provides: parent ? parent.provides : {}, // 获取 parent 的 provides 作为当前组件的初始化值 这样就可以继
承 parent.provides 的属性了
9      proxy: null,
10     isMounted: false,
11     attrs: {}, // 存放 attrs 的数据
12     slots: {}, // 存放插槽的数据
13     ctx: {}, // context 对象
14     setupState: {}, // 存储 setup 的返回值
15     emit: () => {},
16   };
17
18   // 在 prod 环境下的 ctx 只是下面简单的结构
19   // 在 dev 环境下会更复杂
20   instance.ctx = {
21     _: instance,
22   };
23 }
```

```

24 // 赋值 emit
25 // 这里使用 bind 把 instance 进行绑定
26 // 后面用户使用的时候只需要给 event 和参数即可
27 instance.emit = emit.bind(null, instance) as any;
28
29 return instance;
30 }
31

```

- createApp

Code block

```

1 import { createVNode } from "../vnode";
2
3 export function createAppAPI(render) {
4   return function createApp(rootComponent) {
5     const app = {
6       _component: rootComponent,
7       mount(rootContainer) {
8         console.log("基于根组件创建 vnode");
9         const vnode = createVNode(rootComponent);
10        console.log("调用 render, 基于 vnode 进行开箱");
11        render(vnode, rootContainer);
12      },
13    };
14
15    return app;
16  };
17 }
18

```

- 创建Vnode节点

Code block

```

1 import { createVNode } from "../vnode";
2 export const h = (type: any, props: any = null, children: string | Array<any> = []) => {
3   return createVNode(type, props, children);
4 };

```

- 入口文件

Code block

```

1 export * from "./h";
2 export * from "./createApp";
3 export { getCurrentInstance, registerRuntimeCompiler } from "./component";
4 export { inject, provide } from "./apiInject";
5 export { renderSlot } from "./helpers/renderSlot";
6 export { createTextVNode, createElementVNode } from "../vnode";
7 export { createRenderer } from "./renderer";
8 export { toDisplayString } from "@mini-vue/shared";
9 export {
10   // core
11   reactive,
12   ref,
13   readonly,
14   // utilities

```

```

15     unRef,
16     proxyRefs,
17     isReadonly,
18     isReactive,
19     isProxy,
20     isRef,
21     // advanced
22     shallowReadonly,
23     // effect
24     effect,
25     stop,
26     computed,
27 } from "@mini-vue/reactivity";

```

- render

Code block

```

1  // 具体update的Diff见下节课内容;
2  function updateElement(n1, n2, container, anchor, parentComponent) {
3      const oldProps = (n1 && n1.props) || {};
4      const newProps = n2.props || {};
5      // 应该更新 element
6      console.log("应该更新 element");
7      console.log("旧的 vnode", n1);
8      console.log("新的 vnode", n2);
9
10     // 需要把 el 挂载到新的 vnode
11     const el = (n2.el = n1.el);
12
13     // 对比 props
14     patchProps(el, oldProps, newProps);
15
16     // 对比 children
17     patchChildren(n1, n2, el, anchor, parentComponent);
18 }

```

- scheduler

Code block

```

1  // 具体的调度机制见下节课内容
2  const queue: any[] = [];
3  const activePreFlushCbs: any = [];
4
5  const p = Promise.resolve();
6  let isFlushPending = false;
7
8  export function nextTick(fn?) {
9      return fn ? p.then(fn) : p;
10 }
11
12 export function queueJob(job) {
13     if (!queue.includes(job)) {
14         queue.push(job);
15         // 执行所有的 job
16         queueFlush();
17     }
18 }
19

```

```

20 function queueFlush() {
21   // 如果同时触发了两个组件的更新的话
22   // 这里就会触发两次 then （微任务逻辑）
23   // 但是着是没有必要的
24   // 我们只需要触发一次即可处理完所有的 job 调用
25   // 所以需要判断一下 如果已经触发过 nextTick 了
26   // 那么后面就不需要再次触发一次 nextTick 逻辑了
27   if (isFlushPending) return;
28   isFlushPending = true;
29   nextTick(flushJobs);
30 }
31
32 export function queuePreFlushCb(cb) {
33   queueCb(cb, activePreFlushCbs);
34 }
35
36 function queueCb(cb, activeQueue) {
37   // 直接添加到对应的列表内就ok
38   // todo 这里没有考虑 activeQueue 是否已经存在 cb 的情况
39   // 然后在执行 flushJobs 的时候就可以调用 activeQueue 了
40   activeQueue.push(cb);
41
42   // 然后执行队列里面所有的 job
43   queueFlush()
44 }
45
46 function flushJobs() {
47   isFlushPending = false;
48
49   // 先执行 pre 类型的 job
50   // 所以这里执行的job 是在渲染前的
51   // 也就意味着执行这里的 job 的时候 页面还没有渲染
52   flushPreFlushCbs();
53
54   // 这里是执行 queueJob 的
55   // 比如 render 渲染就是属于这个类型的 job
56   let job;
57   while ((job = queue.shift())) {
58     if (job) {
59       job();
60     }
61   }
62 }
63
64 function flushPreFlushCbs() {
65   // 执行所有的 pre 类型的 job
66   for (let i = 0; i < activePreFlushCbs.length; i++) {
67     activePreFlushCbs[i]();
68   }
69 }
70

```

- vnode类型定义及格式规范

Code block

```

1 import { ShapeFlags } from "@mini-vue/shared";
2
3 export { createVNode as createElementVNode }
4
5 export const createVNode = function (

```

```

6     type: any,
7     props?: any,
8     children?: string | Array<any>
9 ) {
10    // 注意 type 有可能是 string 也有可能是对象
11    // 如果是对象的话, 那么就是用户设置的 options
12    // type 为 string 的时候
13    // createVNode("div")
14    // type 为组件对象的时候
15    // createVNode(App)
16    const vnode = {
17      el: null,
18      component: null,
19      key: props?.key,
20      type,
21      props: props || {},
22      children,
23      shapeFlag: getShapeFlag(type),
24    };
25
26    // 基于 children 再次设置 shapeFlag
27    if (Array.isArray(children)) {
28      vnode.shapeFlag |= ShapeFlags.ARRAY_CHILDREN;
29    } else if (typeof children === "string") {
30      vnode.shapeFlag |= ShapeFlags.TEXT_CHILDREN;
31    }
32
33    normalizeChildren(vnode, children);
34
35    return vnode;
36  };
37
38  export function normalizeChildren(vnode, children) {
39    if (typeof children === "object") {
40      // 暂时主要是为了标识出 slots_children 这个类型来
41      // 暂时我们只有 element 类型和 component 类型的组件
42      // 所以我们这里除了 element, 那么只要是 component 的话, 那么children 肯定就是 slots 了
43      if (vnode.shapeFlag & ShapeFlags.ELEMENT) {
44        // 如果是 element 类型的话, 那么 children 肯定不是 slots
45      } else {
46        // 这里就必然是 component 了,
47        vnode.shapeFlag |= ShapeFlags.SLOTS_CHILDREN;
48      }
49    }
50  }
51  // 用 symbol 作为唯一标识
52  export const Text = Symbol("Text");
53  export const Fragment = Symbol("Fragment");
54
55  /**
56   * @private
57   */
58  export function createTextVNode(text: string = " ") {
59    return createVNode(Text, {}, text);
60  }
61
62  // 标准化 vnode 的格式
63  // 其目的是为了 child 支持多种格式
64  export function normalizeVNode(child) {
65    // 暂时只支持处理 child 为 string 和 number 的情况
66    if (typeof child === "string" || typeof child === "number") {
67      return createVNode(Text, null, String(child));

```



```
68   } else {
69     return child;
70   }
71 }
72
73 // 基于 type 来判断是什么类型的组件
74 function getShapeFlag(type: any) {
75   return typeof type === "string"
76     ? ShapeFlags.ELEMENT
77     : ShapeFlags.STATEFUL_COMPONENT;
78 }
79
```

3.4 runtime-dom

Vue3靠虚拟dom，实现跨平台的能力，runtime-dom提供一个渲染器，这个渲染器可以渲染虚拟dom节点到指定的容器中；

3.4.1 主要功能

Code block

```
1  // 源码里面这些接口是由 runtime-dom 来实现
2  // 这里先简单实现
3
4  import { isOn } from "@mini-vue/shared";
5  import { createRenderer } from "@mini-vue/runtime-core";
6
7  // 后面也修改成和源码一样的实现
8  function createElement(type) {
9    console.log("CreateElement", type);
10    const element = document.createElement(type);
11    return element;
12  }
13
14  function createText(text) {
15    return document.createTextNode(text);
16  }
17
18  function setText(node, text) {
19    node.nodeValue = text;
20  }
21
22  function setElementText(el, text) {
23    console.log("SetElementText", el, text);
24    el.textContent = text;
25  }
26
27  function patchProp(el, key, preValue, nextValue) {
28    // preValue 之前的值
29    // 为了之后 update 做准备的值
30    // nextValue 当前的值
31    console.log(`PatchProp 设置属性:${key} 值:${nextValue}`);
32    console.log(`key: ${key} 之前的值是:${preValue}`);
33
34    if (isOn(key)) {
35      // 添加事件处理函数的时候需要注意一下
36      // 1. 添加的和删除的必须是一个函数，不然的话 删除不掉
37      //    那么就需要把之前 add 的函数给存起来，后面删除的时候需要用到
38      // 2. nextValue 有可能是匿名函数，当对比发现不一样的时候也可以通过缓存的机制来避免注册多次
39      // 存储所有的事件函数
40      const invokers = el._vei || (el._vei = {});
```

```
41     const existingInvoker = invokers[key];
42     if (nextValue && existingInvoker) {
43       // patch
44       // 直接修改函数的值即可
45       existingInvoker.value = nextValue;
46     } else {
47       const eventName = key.slice(2).toLowerCase();
48       if (nextValue) {
49         const invoker = (invokers[key] = nextValue);
50         el.addEventListener(eventName, invoker);
51       } else {
52         el.removeEventListener(eventName, existingInvoker);
53         invokers[key] = undefined;
54       }
55     }
56   } else {
57     if (nextValue === null || nextValue === "") {
58       el.removeAttribute(key);
59     } else {
60       el.setAttribute(key, nextValue);
61     }
62   }
63 }
64
65 function insert(child, parent, anchor = null) {
66   console.log("Insert");
67   parent.insertBefore(child, anchor);
68 }
69
70 function remove(child) {
71   const parent = child.parentNode;
72   if (parent) {
73     parent.removeChild(child);
74   }
75 }
76
77 let renderer;
78
79 function ensureRenderer() {
80   // 如果 renderer 有值的话, 那么以后都不会初始化了
81   return (
82     renderer ||
83     (renderer = createRenderer({
84       createElement,
85       createText,
86       setText,
87       setElementText,
88       patchProp,
89       insert,
90       remove,
91     })))
92   );
93 }
94
95 export const createApp = (...args) => {
96   return ensureRenderer().createApp(...args);
97 };
98
99 export * from "@vue/runtime-core"
```

100

3.5 runtime-test

可以理解成runtime-dom的延伸，,因为runtime-test对外提供的确实是dom环境的测试，方便用于runtime-core的测试；

3.5.1 目录结构

Code block

```
1  —src
2  index.ts
3  nodeOps.ts
4  patchProp.ts
5  serialize.ts
6
```

3.5.2 runtime-test核心逻辑

- index.ts

Code block

```
1  // 实现 render 的渲染接口
2  // 实现序列化
3  import { createRenderer } from "@mini-vue/runtime-core";
4  import { extend } from "@vue/shared";
5  import { nodeOps } from "../nodeOps";
6  import { patchProp } from "../patchProp";
7
8  export const { render } = createRenderer(extend({ patchProp }, nodeOps));
9
10 export * from "../nodeOps";
11 export * from "../serialize"
12 export * from '@mini-vue/runtime-core'
```

- nodeOps，节点定义及操作再runtime-core中的映射

Code block

```
1  export const enum NodeTypes {
2    ELEMENT = "element",
3    TEXT = "TEXT",
4  }
5
6  let nodeId = 0;
7  // 这个函数会在 runtime-core 初始化 element 的时候调用
8  function createElement(tag: string) {
9    // 如果是基于 dom 的话 那么这里会返回 dom 元素
10   // 这里是为了测试 所以只需要反正一个对象就可以了
11   // 后面的话 通过这个对象来做测试
12   const node = {
13     tag,
14     id: nodeId++,
15     type: NodeTypes.ELEMENT,
16     props: {},
17     children: [],
18     parentNode: null,
19   };
20
21   return node;
22 }
23
```

```

24 function insert(child, parent) {
25   parent.children.push(child);
26   child.parentNode = parent;
27 }
28
29 function parentNode(node) {
30   return node.parentNode;
31 }
32
33 function setElementText(el, text) {
34   el.children = [
35     {
36       id: nodeId++,
37       type: NodeTypes.TEXT,
38       text,
39       parentNode: el,
40     },
41   ];
42 }
43
44 export const nodeOps = { createElement, insert, parentNode, setElementText };
45

```

- serialize, 序列化：把Vnode处理成 string

Code block

```

1  // 把 node 给序列化
2  // 测试的时候好对比
3
4  import { NodeTypes } from './nodeOps';
5
6  // 序列化： 把一个对象给处理成 string （进行流化）
7  export function serialize(node) {
8    if (node.type === NodeTypes.ELEMENT) {
9      return serializeElement(node);
10   } else {
11     return serializeText(node);
12   }
13 }
14
15 function serializeText(node) {
16   return node.text;
17 }
18
19 export function serializeInner(node) {
20   // 把所有节点变成一个string
21   return node.children.map((c) => serialize(c)).join('');
22 }
23
24 function serializeElement(node) {
25   // 把 props 处理成字符串
26   // 规则：
27   // 如果 value 是 null 的话 那么直接返回 ``
28   // 如果 value 是 `` 的话，那么返回 key
29   // 不然的话返回 key = value (这里的值需要字符串化)
30   const props = Object.keys(node.props)
31     .map((key) => {
32       const value = node.props[key];
33       return value == null
34         ? ``

```

```

35         : value === ``
36         ? key
37         : `${key}=${JSON.stringify(value)}`;
38     })
39     .filter(Boolean)
40     .join(" ");
41
42     console.log("node-----", node.children);
43     return `<${node.tag}${props ? ` ${props}` : ``}>${serializeInner(node)}</${
44         node.tag
45     }>`;
46 }
47

```

3.6 shared

公用逻辑

3.6.1 具体逻辑

Code block

```

1  export * from '../src/shapeFlags';
2  export * from '../src/toDisplayString';
3
4  export const isObject = val => {
5      return val !== null && typeof val === 'object';
6  };
7
8  export const isString = val => typeof val === 'string';
9
10 const camelizeRE = /-(\w)/g;
11 /**
12  * @private
13  * 把中划线命名方式转换成驼峰命名方式
14  */
15 export const camelize = (str: string): string => {
16     return str.replace(camelizeRE, (_, c) => (c ? c.toUpperCase() : ''));
17 };
18
19 export const extend = Object.assign;
20
21 // 必须是 on+一个大写字母的格式开头
22 export const isOn = key => /^on[A-Z]/.test(key);
23
24 export function hasChanged(value, oldValue) {
25     return !Object.is(value, oldValue);
26 }
27
28 export function hasOwn(val, key) {
29     return Object.prototype.hasOwnProperty.call(val, key);
30 }
31
32 /**
33  * @private
34  * 首字母大写
35  */
36 export const capitalize = (str: string) => str.charAt(0).toUpperCase() + str.slice(1);
37
38 /**
39  * @private

```

```
40  * 添加 on 前缀, 并且首字母大写
41  */
42  export const toHandlerKey = (str: string) => (str ? `on${capitalize(str)}` : ``);
43
44  // 用来匹配 kebab-case 的情况
45  // 比如 onTest-event 可以匹配到 T
46  // 然后取到 T 在前面加一个 - 就可以
47  // \BT 就可以匹配到 T 前面是字母的位置
48  const hyphenateRE = /\B([A-Z])/g;
49  /**
50   * @private
51   */
52  export const hyphenate = (str: string) => str.replace(hyphenateRE, '-$1').toLowerCase();
53
54
55  // 组件的类型
56  export const enum ShapeFlags {
57    // 最后要渲染的 element 类型
58    ELEMENT = 1,
59    // 组件类型
60    STATEFUL_COMPONENT = 1 << 2,
61    // vnode 的 children 为 string 类型
62    TEXT_CHILDREN = 1 << 3,
63    // vnode 的 children 为数组类型
64    ARRAY_CHILDREN = 1 << 4,
65    // vnode 的 children 为 slots 类型
66    SLOTS_CHILDREN = 1 << 5
67  }
68
69  export const toDisplayString = (val) => {
70    return String(val);
71  };
72
```

4. 9~10月如何平稳收获offer?



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7





5. Vue 工程化 & 性能优化：如何基于Vite实现性能优化

5.1 包体积与 Tree-shaking 优化

一个最有效的提升页面加载速度的方法就是压缩 JavaScript 打包产物的体积。当使用 Vue 时有下面一些办法来减小打包产物体积：

- 尽可能地采用构建步骤
 - 如果使用的是相对现代的打包工具，许多 Vue 的 API 都是可以被 `tree-shake` 的。举例来说，如果你根本没有使用到内置的 `<Transition>` 组件，它将不会被打包进入最终的产物里。Tree-shaking 也可以移除你源代码中其他未使用到的模块。
 - 当使用了构建步骤时，模板会被预编译，因此我们无须在浏览器中载入 Vue 编译器。这在同样最小化加上 gzip 优化下会相对缩小 14kb 并避免运行时的编译开销。
- 在引入新的依赖项时要小心包体积膨胀！在现实的应用中，包体积膨胀通常因为无意识地引入了过重的依赖导致的。
 - 如果使用了构建步骤，应当尽量选择提供 ES 模块格式的依赖，它们对 tree-shaking 更友好。举例来说，选择 `lodash-es` 比 `lodash` 更好。
 - 查看依赖的体积，并评估与它所提供的功能之间的性价比。如果依赖对 tree-shaking 友好，实际增加的体积大小将取决于你从它之中导入的 API。像 `bundlejs.com` 这样的工具可以用来做快速的检查，但是根据实际的构建设置来评估总是最准确的。

5.2 代码分割

代码分割是指构建工具将构建后的 JavaScript 包拆分为多个较小的，可以按需或并行加载的文件。通过适当的代码分割，页面加载时需要的功能可以立即下载，而额外的块只在需要时才加载，从而提高性能。

像 Rollup (Vite 就是基于它之上开发的) 或者 webpack 这样的打包工具可以通过分析 ESM 动态导入的语法来自动进行代码分割：

Code block

```
1  // lazy.js 及其依赖会被拆分到一个单独的文件中
2  // 并只在 `loadLazy()` 调用时才加载
3  function loadLazy() {
4    return import('./lazy.js')
5  }
```

懒加载对于页面初次加载时的优化帮助极大，它帮助应用暂时略过了那些不是立即需要的功能。在 Vue 应用中，这可以与 Vue 的[异步组件](#)搭配使用，为组件树创建分离的代码块：

Code block

```
1  import { defineAsyncComponent } from 'vue'
2
3  // 会为 Foo.vue 及其依赖创建单独的一个块
4  // 它只会按需加载
5  // （即该异步组件在页面中被渲染时）
6  const Foo = defineAsyncComponent(() => import('./Foo.vue'))
```

5.3 更新优化

5.3.1 Props 稳定性

在 Vue 之中，一个子组件只会在其至少一个 props 改变时才会更新。思考以下示例：

Code block

```
1  <ListItem
2    v-for="item in list"
3    :id="item.id"
4    :active-id="activeId" />
```

在 `<ListItem>` 组件中，它使用了 `id` 和 `activeId` 两个 props 来确定它是否是当前活跃的那一项。虽然这是可行的，但问题是每当 `activeId` 更新时，列表中的每一个 `<ListItem>` 都会跟着更新！

理想情况下，只有活跃状态发生改变的项才应该更新。我们可以将活跃状态比对的逻辑移入父组件来实现这一点，然后让 `<ListItem>` 改为接收一个 `active` prop：

Code block

```
1  <ListItem
2    v-for="item in list"
3    :id="item.id"
4    :active="item.id === activeId" />
```

5.3.2 `v-once`

`v-once` 是一个内置的指令，可以用来渲染依赖运行时数据但无需再更新的内容。它的整个子树都会在未来的更新中被跳过。

在随后的重新渲染，元素/组件及其所有子项将被当作静态内容并跳过渲染。这可以用来优化更新时的性能。

Code block

```
1  <!-- 单个元素 -->
2  <span v-once>This will never change: {{msg}}</span>
3  <!-- 带有子元素的元素 -->
```

```
4   <div v-once>
5     <h1>comment</h1>
6     <p>{{msg}}</p>
7   </div>
8   <!-- 组件 -->
9   <MyComponent v-once :comment="msg" />
10  <!-- `v-for` 指令 -->
11  <ul>
12    <li v-for="i in list" v-once>{{i}}</li>
13  </ul>
```

5.3.3 v-memo

`v-memo` 是一个内置指令，可以用来有条件地跳过某些大型子树或者 `v-for` 列表的更新。

缓存一个模板的子树。在元素和组件上都可以使用。为了实现缓存，该指令需要传入一个固定长度的依赖值数组进行比较。如果数组里的每个值都与最后一次的渲染相同，那么整个子树的更新将被跳过。举例来说：

template

Code block

```
1   <div v-memo="[valueA, valueB]">
2     ...
3   </div>
```

当组件重新渲染，如果 `valueA` 和 `valueB` 都保持不变，这个 `<div>` 及其子项的所有更新都将被跳过。实际上，甚至虚拟 DOM 的 vnode 创建也将被跳过，因为缓存的子树副本可以被重新使用。

正确指定缓存数组很重要，否则应该生效的更新可能被跳过。`v-memo` 传入空依赖数组 (`v-memo="[]"`) 将与 `v-once` 效果相同。

与 `v-for` 一起使用

`v-memo` 仅用于性能至上场景中的微小优化，应该很少需要。最常见的情况可能是有助于渲染海量 `v-for` 列表 (长度超过 1000 的情况)：

template

Code block

```
1   <div v-for="item in list" :key="item.id" v-memo="[item.id === selected]">
2     <p>ID: {{ item.id }} - selected: {{ item.id === selected }}</p>
3     <p>...more child nodes</p>
4   </div>
```

当组件的 `selected` 状态改变，默认会重新创建大量的 vnode，尽管绝大部分都跟之前是一模一样的。`v-memo` 用在这里本质上是在说“只有当该项的被选中状态改变时才需要更新”。这使得每个选中状态没有变的项能完全重用之前的 vnode 并跳过差异比较。注意这里 memo 依赖数组中并不需要包含 `item.id`，因为 Vue 也会根据 item 的 `:key` 进行判断。

5.4 通用优化

5.4.1 虚拟滚动

根据可视区的高度以及items中每一项的高度(itemSize，可为高度或者是横向滑动的宽度)来决定页面展示多少个item，能显示的item包装后放到了pool数组中进行渲染，页面滚动的时候动态的修改pool数组。为了在滚动的时候尽可能的减少开销，pool中超出范围的view会回收到复用池，pool中新增的view会优先从复用池中取出view，如果没有复用的才会新增。

5.4.2 减少大型不可变数据的响应性开销

Vue 的响应性系统默认是深度的。虽然这让状态管理变得更直观，但在数据量巨大时，深度响应性也会导致不小的性能负担，因为每个属性访问都将触发代理的依赖追踪。好在这种性能负担通常只有在处理超大型数组或层级很深的对象时，例如一次渲染需要访问 100,000+ 个属性时，才会变得比较明显。因此，它只会影响少数特定的场景。

Vue 确实也为此提供了一种解决方案，通过使用 `shallowRef()` 和 `shallowReactive()` 来绕开深度响应。浅层式 API 创建的状态只在其顶层是响应式的，对所有深层的对象不会做任何处理。这使得对深层级属性的访问变得更快，但代价是，我们现在必须将所有深层级对象视为不可变的，并且只能通过替换整个根状态来触发更新：

Code block

```
1  const shallowArray = shallowRef([
2    /* 巨大的列表，里面包含深层的对象 */
3  ])
4
5  // 这不会触发更新...
6  shallowArray.value.push(newObject)
7  // 这才会触发更新
8  shallowArray.value = [...shallowArray.value, newObject]
9
10 // 这不会触发更新...
11 shallowArray.value[0].foo = 1
12 // 这才会触发更新
13 shallowArray.value = [
14   {
15     ...shallowArray.value[0],
16     foo: 1
17   },
18   ...shallowArray.value.slice(1)
19 ]
```

5.4.3 避免不必要的组件抽象

有些时候我们会去创建[无渲染组件](#)或高阶组件 (用来渲染具有额外 props 的其他组件) 来实现更好的抽象或代码组织。虽然这并没有什么问题，但请记住，组件实例比普通 DOM 节点要昂贵得多，而且为了逻辑抽象创建太多组件实例将会导致性能损失。

需要提醒的是，只减少几个组件实例对于性能不会有明显的改善，所以如果一个用于抽象的组件在应用中只会渲染几次，就不用操心去优化它了。考虑这种优化的最佳场景还是在大型列表中。想象一下一个有 100 项的列表，每项的组件都包含许多子组件。在这里去掉一个不必要的组件抽象，可能会减少数百个组件实例的无谓性能消耗。

6. Vue 最佳实践：基于Vue3的最佳实践详解

 encode-chatgpt-vue.zip

370.11KB



详情见课上内容